

23. S-атрибутные грамматики. Реализация при восходящем анализе.

Атрибутная грамматика для синтаксического дерева

Для КС-грамматики арифметических выражений построим атрибутную грамматику, которая будет строить синтаксическое дерево одновременно с выводом цепочки.

- Синтезируемый атрибут *ptr* у нетерминалов используется для хранения указателя на узел синтаксического дерева, соответствующий данному нетерминалу.
- Функция `MKLEAF(a, entry)` создает лист, помеченный операндом *a* и содержащий значение атрибута *entry* – указателя на запись в таблице символов.
- Функция `MKNODE(op, left, right)` создает узел операции, помеченный оператором *op* и содержащий указатели *left* и *right* на узлы операндов.

S-атрибутная грамматика

Определение

Атрибутная грамматика, имеющая только синтезируемые атрибуты, называется *S-атрибутой*.

Естественный порядок вычисления таких атрибутов – снизу вверх по дереву вывода – совпадает с порядком построения дерева вывода при восходящем анализе.

Напомним, что в стеке LR-анализатора хранятся не грамматические символы, а *состояния*, содержащие информацию об уже разобранных поддеревьях дерева вывода, при этом символ однозначно определяется состоянием.

Восходящий анализ S-атрибутной грамматики

Для семантического анализа элемент стека делают записью, имеющей синтаксическое поле $state$ (состояние) и семантическое поле val (значение синтезируемого атрибута соответствующего символа). Если атрибутов несколько, то val рассматривается как вектор. Через $state[top]$ ($val[top]$) мы будем обозначать соответствующие значения для верхней записи в стеке; через $state[top-i]$ ($val[top-i]$) – значения для $(i+1)$ -й сверху записи.

Шаг работы анализатора начинается с вычисления значения указателя на новую вершину стека ($ntop$). Условно говоря, что при переносе выполняется присваивание $ntop = top+1$, а при свертке по правилу вывода с r символами в правой части – присваивание $ntop = top-r+1$. Заканчивается шаг присваиванием $top = ntop$.

Navigation icons: back, forward, search, etc.

Действия в стеке LR-анализатора

Если на данном шаге выполняется перенос терминала b , то анализатор присвоит $state[top] = b$ и $val[top] = b.lexval$. Пусть перед данным шагом стек выглядит как в таблице

$state$	val
Z	$Z.z$
Y	$Y.y$
X	$X.x$
$\dots$	$\dots$

$\leftarrow top$

и на шаге выполняется
свертка по правилу $A \rightarrow XYZ$
с вычислением синтезируемого
атрибута $A.a = f(X.x, Y.y, Z.z)$.

Тогда анализатор присвоит
 $state[ntop] = A$,

$val[ntop] =$
 $f(val[top-2], val[top-1], val[top])$

и выполнит какие-то обусловленные

конкретной реализацией действия.

Navigation icon: search.

Navigation icons: back, forward, search, etc.

Пример

Возьмем атрибутивную грамматику для вычисления арифметических выражений и добавим к ней внешнюю аксиому E' . В таблице семантические действия приведены в виде фрагментов кода, выполняемых LR-анализатором *перед* соответствующей сверткой. Пустые клетки в таблице означают, что никаких действий производить не требуется, т. е. $ntop = top$ и значение $val[ntop]$ уже находится в поле $val[top]$.

	Правило вывода	Фрагмент кода
1	$E' \rightarrow E$	<code>PRINT(val[top])</code>
2	$E \rightarrow E_1 + T$	<code>val[ntop] = val[top-2] + val[top]</code>
3	$E \rightarrow T$	
4	$T \rightarrow T_1 * F$	<code>val[ntop] = val[top-2] * val[top]</code>
5	$T \rightarrow F$	
6	$F \rightarrow (E)$	<code>val[ntop] = val[top-1]</code>
7	$F \rightarrow x$	

1. Основные определения

S-атрибутивная грамматика — это тип атрибутивной грамматики, в которой используются только **синтезируемые атрибуты**.

- **Синтезируемый атрибут** — это атрибут, значение которого в узле дерева вычисляется через значения атрибутов его дочерних узлов.
- **Особенность:** Порядок вычисления таких атрибутов — **снизу вверх** (от листьев к корню). Это идеально совпадает с порядком работы восходящих (bottom-up) парсеров, таких как LR-анализаторы.

Детальное управление указателем стека

Для реализации семантики анализатор использует два указателя: `top` (текущая вершина) и `ntop` (новая вершина).

- **Доступ к данным:** `* val[top]` — атрибут верхнего элемента.
 - `val[top-i]` — атрибут (i+1)-го элемента сверху.
- **Логика ntop:**
 - При **переносе (Shift)**: `ntop = top + 1`. Стек просто растет вверх.
 - При **свертке (Reduce)** по правилу с длиной правой части `r`: `ntop = top - r + 1`.
Это позволяет записать результат вычисления прямо в ту позицию, где начиналась правая часть правила, эффективно заменяя её одним нетерминалом.
- **Завершение:** В конце каждого шага выполняется присваивание `top = ntop`.

2. Механизм реализации в LR-анализаторе

В стандартном LR-анализаторе в стеке хранятся состояния (`state`). Для реализации S-атрибутивной грамматики каждая запись в стеке расширяется полем `val` для хранения значения атрибута.

Структура стека

Стек представляет собой массив записей, где каждая запись — это пара (`state`, `val`).

- `state` — состояние синтаксического анализатора.

- `val` — значение синтезируемого атрибута для соответствующего символа.

Управление указателями стека

При выполнении шага анализатора вычисляется новое положение вершины стека — `ntop` :

1. **Перенос (Shift):** `ntop = top + 1` . Атрибут терминала (например, значение числа из лексера) записывается в `val[ntop]` .
2. **Свертка (Reduce):** Если применяется правило $A \rightarrow XYZ$ (где $r = 3$ символа в правой части):
 - `ntop = top - r + 1` .
 - Значение атрибута нетерминала A вычисляется как функция от значений атрибутов X, Y, Z , которые уже лежат в стеке на позициях `top-2` , `top-1` и `top` .

3. Функции построения синтаксического дерева

Если атрибутом является не число, а указатель на узел дерева (`ptr`), используются следующие функции:

- **MKLEAF(`id`, `entry`)** : Создает лист дерева для операнда (идентификатора или константы). `entry` — указатель на запись в таблице символов.
- **MKNODE(`op`, `left`, `right`)** : Создает внутренний узел для операции `op` с указателями на левое и правое поддеревья.

4. Подробный разбор примера

Рассмотрим грамматику арифметических выражений, дополненную семантическими действиями для вычисления значений.

№	Правило вывода	Фрагмент кода (Семантическое действие)	Комментарий
1	$E' \rightarrow E$	<code>PRINT(val[top])</code>	Вывод финального результата в корне.
2	$E \rightarrow E_1 + T$	<code>val[ntop] = val[top-2] + val[top]</code>	Свертка суммы: E_1 на <code>top-2</code> , знак <code>+</code> на <code>top-1</code> , T на <code>top</code> .
3	$E \rightarrow T$	(пусто)	Значение T просто переходит к E (<code>ntop</code> совпадает с <code>top</code>).
4	$T \rightarrow T_1 * F$	<code>val[ntop] = val[top-2] * val[top]</code>	Свертка произведения.
5	$T \rightarrow F$	(пусто)	Передача значения вверх.
6	$F \rightarrow (E)$	<code>val[ntop] = val[top-1]</code>	Значение выражения E берется с позиции <code>top-1</code> , т.к. на <code>top</code> — <code>)</code> .
7	$F \rightarrow x$	(пусто)	Значение <code>x</code> уже лежит в <code>val[top]</code> после фазы переноса.

Пример процесса для выражения `3 * 5` :

1. **Shift 3** : В стеке `val[top] = 3` .
2. **Reduce $F \rightarrow x, T \rightarrow F$** : Значение `3` поднимается по стеку.
3. **Shift *** : Стек растёт.
4. **Shift 5** : В стеке `val[top] = 5` .
5. **Reduce $T \rightarrow T * F$** :
 - Анализатор видит в стеке: `[3]` (на `top-2`), `[*]` (на `top-1`), `[5]` (на `top`).
 - Выполняет: `3 * 5` .
 - Результат `15` записывается в `val[ntop]` , заменяя собой всю правую часть правила в стеке.

Итог: Реализация S-атрибутных грамматик в восходящих анализаторах эффективна, так как не требует отдельного обхода дерева — вычисления происходят параллельно с синтаксическим анализом.